

CampusRide — Design Document

Real-Time Campus Mobility & Ride Management Platform

Cult Open Projects 2027 · Software Development | Stack: MERN + Socket.IO

1. Problem Understanding

Large campuses such as IIT Roorkee depend on e-rickshaws for last-mile transport, but ride requests and driver availability are coordinated through fragmented, informal mechanisms. This causes inefficient resource use, delays, uneven demand distribution and a poor experience for both riders and drivers.

Goal: a centralized, real-time platform where passengers and drivers connect to *discover, request, assign, track and complete* rides reliably within a campus.

The engineering challenges mirrored from production ride-hailing systems are:

- **Real-time communication** — both sides must see state changes instantly.
- **Consistent ride state** — one well-defined lifecycle; a request assigned to **exactly one** driver.
- **Multi-user coordination** — many passengers and drivers acting concurrently.
- **Geospatial handling** — pickup, destination and driver positions on a map.
- **Insight** — dashboards and demand analytics for drivers and operators.

Scope delivered: all seven mandatory features (authentication, availability, request workflow, real-time updates, lifecycle, driver dashboard, ratings) plus two bonus features — **live map** and **demand analytics**. Payments, scheduling and ML forecasting were deliberately scoped out to keep the core workflow reliable, following the guideline that "a reliable ride management system is preferable to numerous incomplete features."

2. System Architecture

CampusRide is a **three-tier MERN application** with a dedicated real-time channel.



Request vs. real-time split

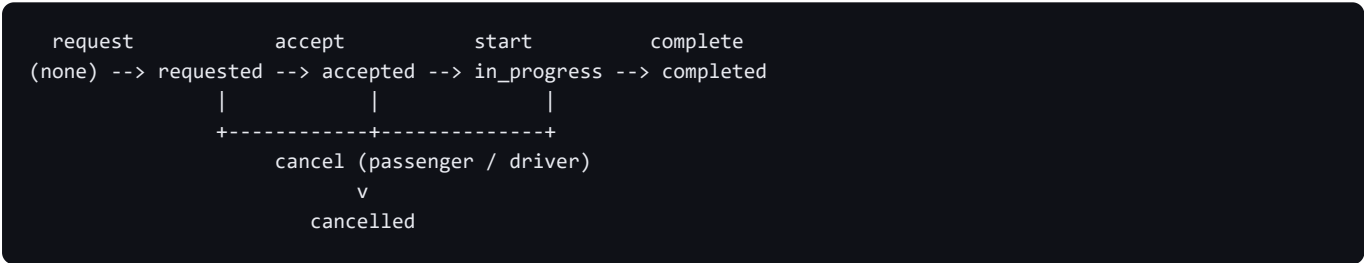
- **REST** is the source of truth for every state mutation (request, accept, start, complete, cancel, rate). Each endpoint validates, persists, then **emits** the result.
- **Socket.IO** only *pushes* those changes to relevant clients — never the sole authority — so state stays consistent even when a socket reconnects.

Socket room model

Room	Members	Used for
user:<id>	one user	personal notifications
drivers	all connected drivers	broadcasting new ride requests
passengers	all connected passengers	driver availability / location feed
ride:<rideId>	ride's passenger + driver	per-ride lifecycle updates

Sockets authenticate with the same JWT during the handshake, so every event is scoped to an authorised user.

Ride lifecycle (state machine)



Transitions are guarded server-side: a ride can only move along allowed edges, and only by an authorised participant.

3. Database Schema

MongoDB document model via Mongoose — three collections.

users

Field	Type	Notes
_id	ObjectId	PK
name	String	required
email	String	required, unique, lowercased
phone	String	
passwordHash	String	bcrypt; never returned (select:false)
role	String	passenger driver
vehicle	Object	(driver) type, model, plateNumber, color, capacity
verification	Object	(driver) licenseNumber, verified
isOnline	Boolean	(driver) availability switch
availabilityStatus	String	available busy offline
currentLocation	Object	lat, lng, label, updatedAt
ratingAvg / ratingCount	Number	(driver) aggregate rating

rides

Field	Type	Notes
_id	ObjectId	PK
passenger	ObjectId → users	required, indexed
driver	ObjectId → users	nullable until accepted, indexed
pickup / destination	Point	{ label, lat, lng }
status	String	requested / accepted / in_progress / completed / cancelled
distanceKm / fare	Number	computed (haversine; base + per-km)
rejectedBy	[ObjectId]	drivers who declined (not re-offered)
requestedAt ... cancelledAt	Date	lifecycle timestamps
cancelledBy	String	passenger driver
rated	Boolean	guards one rating per ride

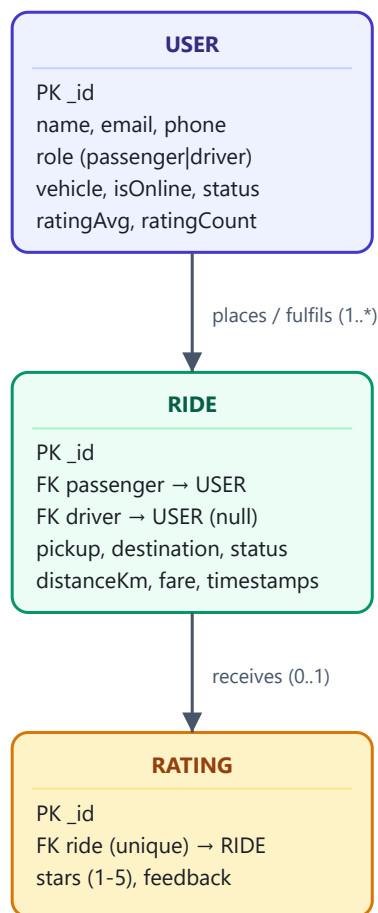
ratings

Field	Type	Notes
_id	ObjectId	PK

ride	ObjectId → rides	unique (one rating per ride)
passenger	ObjectId → users	author
driver	ObjectId → users	subject, indexed
stars	Number	1–5
feedback	String	optional

Indexes: users.email (unique), rides.passenger, rides.driver, rides.status+createdAt, ratings.ride (unique), ratings.driver.

4. Entity Relationship Diagram (ERD)



- A **passenger** places many **rides** (1-to-many).
- A **driver** fulfils many **rides** (a ride has 0-or-1 driver).
- A **ride** receives 0-or-1 **rating**; a rating references its ride, passenger and driver.

5. API Overview

Base URL `/api`. JSON over HTTP; JWT bearer auth; real-time via Socket.IO.

Method	Endpoint	Auth	Description
POST	<code>/auth/register</code>	—	Register (passenger or driver)
POST	<code>/auth/login</code>	—	Login, returns JWT
GET	<code>/auth/me</code>	any	Current user
PATCH	<code>/auth/profile</code>	any	Update profile / vehicle
GET	<code>/drivers/available</code>	any	List online & available drivers
PATCH	<code>/drivers/availability</code>	driver	Go online / offline
PATCH	<code>/drivers/location</code>	driver	Update live location
POST	<code>/rides</code>	passenger	Request a ride
GET	<code>/rides</code>	any	My rides
GET	<code>/rides/requests</code>	driver	Open requests to accept
PATCH	<code>/rides/:id/accept</code>	driver	Accept (atomic single assignment)
PATCH	<code>/rides/:id/{reject,start,complete,cancel}</code>	driver / participant	Lifecycle transitions
POST	<code>/ratings</code>	passenger	Rate a completed ride
GET	<code>/ratings/driver/:id</code>	any	Driver feedback history
GET	<code>/analytics/driver</code>	driver	Driver dashboard metrics
GET	<code>/analytics/demand</code>	any	Campus demand analytics (bonus)

Real-time events: `ride:new`, `ride:assigned`, `ride:updated`, `ride:cancelled`, `ride:requestClosed`, `driver:availability`, `driver:location`.

Example — atomic accept: `findOneAndUpdate({ _id, status:'requested', driver:null }, { driver, status:'accepted' })` → emits `ride:assigned` to the passenger and `ride:requestClosed` to other drivers.

6. Design Decisions

1. **REST for truth, sockets for push.** All mutations go through validated REST endpoints; Socket.IO only broadcasts results — avoiding race conditions and surviving reconnects.
2. **Atomic single-driver assignment.** Acceptance is a single conditional `findOneAndUpdate` (`status:'requested', driver:null`). MongoDB guarantees only one concurrent driver wins; others get a clear "already taken" response — satisfying "a ride can only be assigned to a single driver."
3. **Server-guarded state machine.** Lifecycle transitions are validated server-side (allowed edges + participant/role checks), so the UI can never force an invalid state.
4. **Room-scoped events.** Emitting to `user:`, `ride:`, `drivers`, `passengers` rooms delivers only relevant events — efficient and scalable.

5. **JWT + role middleware.** Stateless auth scales horizontally; `requireRole` centralises authorisation; passwords are bcrypt-hashed and never serialized.
6. **Zero-setup dev experience.** An in-memory MongoDB option with auto-seeding lets evaluators run the full stack with only Node installed; a real `MONGO_URI` is a one-line switch.
7. **Computed fare/distance.** Haversine distance + a transparent base+per-km fare keep estimates deterministic without a paid routing API.
8. **Scalability path.** Stateless API behind a load balancer; Socket.IO scales across nodes via a Redis adapter; indexes on hot query paths keep reads fast as data grows.

Trade-offs & future work

- Matching is **broadcast-to-all-online** (simple, transparent) rather than proximity-ranked dispatch — a geo-nearest assignment is the natural next step.
- Payments, scheduling and ML demand forecasting are scoped out, but the schema (timestamps, locations) already supports adding them.